



Probabilistic Machine Learning

18. Trees, Forests, Bagging and Boosting 박준수 / 2026.04.09



Computational Data Science LAB

CONTENTS

1. Classification and regression trees (CART)
2. Ensemble learning
3. Bagging
4. Random forests
5. Boosting
6. Interpreting tree importance

01 | Classification and Regression Trees (CART)

Model definition & fitting

- 트리 모델 정의

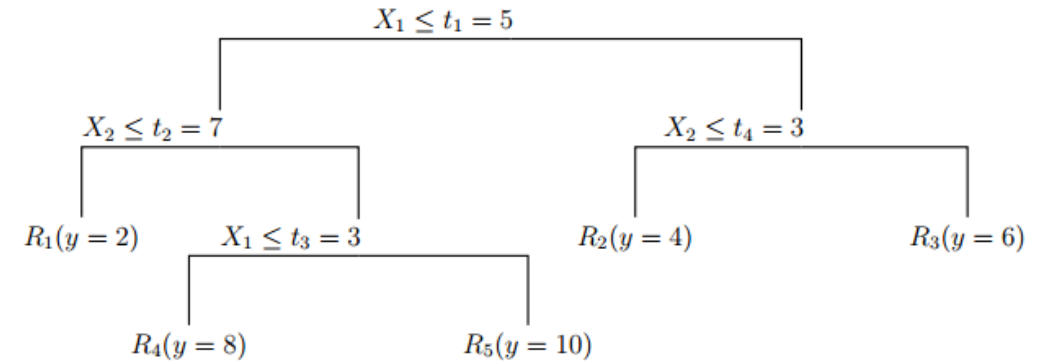
- ✓ 입력 공간을 여러 영역 R_j 로 분할
- ✓ 각 영역에 대표값 w_j 할당하여 예측
- ✓ $f(x) = \sum w_j I(x \in R_j)$

- 분할 방식 (split)

- ✓ Feature x_j 와 threshold t 로 binary split
- ✓ Axis-parallel partition (수직/ 수평 분할)
- ✓ 불순도 감소를 최대화하는 방향으로 분할

- 모델 학습 (Model fitting)

- ✓ 전체 최적화 불가능 $\Rightarrow$ greedy 방식 사용
- ✓ 각 노드에서 최적 feature와 threshold 선택
- ✓ $\frac{|D_L|}{|D|} c(D_L) + \frac{|D_R|}{|D|} c(D_R)$ 최소화



- Cost 함수

- ✓ Regression: MSE
- ✓ Classification: Gini / Entropy

02 | Ensemble Learning

Variance reduction & basic idea

- Ensemble learning
 - ✓ 여러 모델의 예측을 결합하여 성능 향상
 - ✓ 특히 high variance 모델(예: tree)의 불안정성 감소
 - ✓ 예측의 평균을 통해 노이즈를 상쇄
- 기본 구조
 - ✓ Regression $\Rightarrow$ 평균
 - ✓ Classification $\Rightarrow$ majority vote
 - ✓
$$f(x) = \frac{1}{M} \sum f_m(x)$$
- 효과
 - ✓ Bias $\approx$ 유지
 - ✓ Variance $\downarrow$ (평균으로 흔들림 감소)
- 핵심 조건
 - ✓ 모델들이 서로 다르게 틀려야 효과 있음 (diversity)

02 | Ensemble Learning

Stacking & BMA

- Stacking
 - ✓ 여러 모델의 출력값을 새로운 feature로 사용
 - ✓ meta model이 최종 결합 방식 학습
 - ✓ 단순 평균이 아닌, 데이터 기반으로 가중치 학습
- BMA (Bayesian Model Averaging)
 - ✓ 모델 확률 $p(m|D)$ 기반 결합
 - ✓ $p(y|x, D) = \sum p(m|D)p(y|x, m, D)$
 - ✓ 가중치 = $p(m|D)$, 가중치의 합 = 1
- BMA 특징
 - ✓ 데이터 많아질수록 특정 모델에 확률 집중
 - ✓ 최종적으로 단일 모델 선택에 가까워짐

03 | Bagging

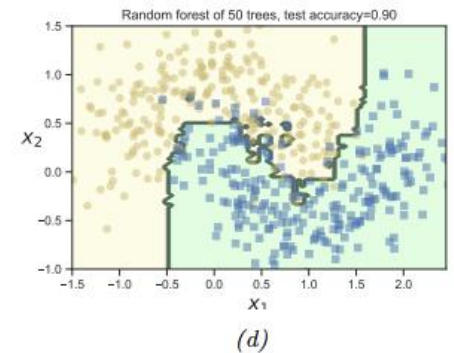
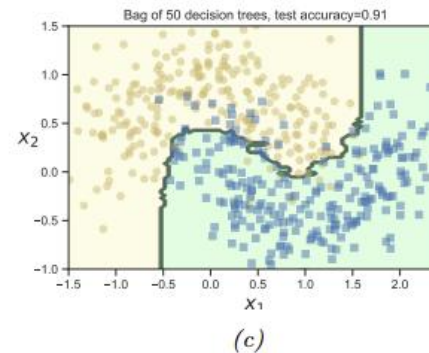
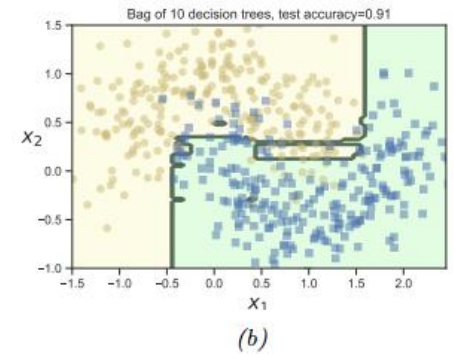
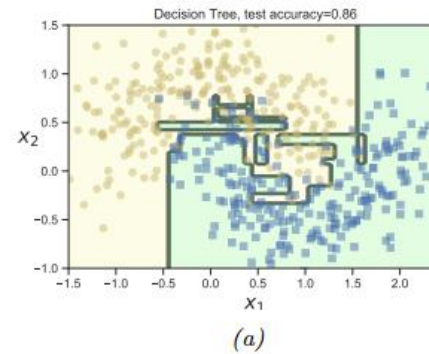
Variance reduction via data sampling

- Bagging 개념
 - ✓ 데이터를 랜덤하게 여러 번 샘플링하여 여러 모델 학습 후 평균
 - ✓ Bootstrap sampling (복원추출) 사용
- 학습 과정
 - ✓ 원 데이터 N개 $\Rightarrow$ 중복 허용하여 N개 재샘플링
 - ✓ 각 bootstrap 샘플로 모델 M개 학습
 - ✓ 최종 예측 $\Rightarrow$ 평균 / majority vote
- 핵심 효과
 - ✓ 데이터 샘플이 달라지면서 모델 다양성 확보
 - ✓ 서로 다른 오차가 평균되며 variance 감소
 - ✓ 모델의 불안정성(특히 tree) 완화
- Out-of-Bag (OOB)
 - ✓ bootstrap sampling 시 약 37% 데이터는 미선택
 - ✓ OOB 데이터로 성능 평가 가능 (CV 대체)

04 | Random Forest

Bagging + random feature selection

- Random Forest
 - ✓ Bagging 기반 tree ensemble
 - ✓ 데이터 + feature 랜덤성 동시 적용
- 학습 방식
 - ✓ Bootstrap sample로 각 tree 학습
 - ✓ 각 node에서 feature subset (S_i) 선택
 - ✓ (S_i) 내에서 최적 feature와 threshold로 split
- 핵심 아이디어
 - ✓ Bagging만 사용시 tree 간 유사성 ↑
 - ✓ Node마다 feature 제한 $\Rightarrow$ 상관관계 ↓
 - ✓ 다양성 증가 $\Rightarrow$ 일반화 성능 향상
- 특징
 - ✓ 과적합 감소
 - ✓ Irrelevant feature 많은 경우 효과적



05 | Boosting

Additive model & problem setting

- Boosting의 기본 구조

- ✓ 여러 weak learner를 순차적으로 학습하여 결합
- ✓ $f(x) = \sum_{m=1}^M \beta_m F_m(x)$
- ✓ 각 F_m 은 단순 모델 (tree, linear model 등)

- 학습 문제 정의

- ✓ 예측 문제를 loss 최소화 문제로 설정
- ✓ $L(f) = \sum \ell(y_i, f(x_i))$
- ✓ 데이터 분포를 모델로 근사하는 과정

- 핵심 특징

- ✓ 모델을 한 번에 학습하지 않고 순차적으로 추가하며 성능 개선
- ✓ bias를 줄이는 방향으로 작동

- Random Forest vs Boosting

- ✓ Random Forest: 독립적 학습 (parallel), variance 감소
- ✓ Boosting: 순차적 학습 (sequential), bias 감소

05 | Boosting

Forward stagewise additive modeling

- 단계적 모델 학습
 - ✓ 전체 모델을 한 번에 최적화하지 않음
 - ✓ 한 번에 하나의 모델 F_m 만 추가
- 업데이트 방식
 - ✓ $f_m(x) = f_{m-1}(x) + \beta_m F_m(x)$
 - ✓ 이전 모델은 고정, 새 모델만 학습
- 최적화 관점
 - ✓ 현재 모델에 새로운 함수 추가 시 loss가 최소가 되도록 F_m, β_m 선택
- 의미
 - ✓ 각 단계는 이전 모델의 틀린 부분을 보완
 - ✓ 점진적으로 전체 예측 성능 개선

05 | Boosting

Least squares boosting & Adaboost

- Least squares boosting (Squared loss)

- ✓ loss: $(y - f(x))^2$
- ✓ residual: $y - f(x)$
- ✓ 새로운 모델이 residual을 직접 예측

- AdaBoost (Exponential loss)

- ✓ $p(y = 1|x) = \frac{e^{F(x)}}{e^{-F(x)} + e^{F(x)}} = \frac{1}{1 + e^{-2F(x)}}$
- ✓ loss: $e^{-\tilde{y}f(x)}$
- ✓ 틀린 데이터에 높은 weight 부여
- ✓ Weighted dataset 기반 학습

- 핵심 차이

- ✓ Least squares boosting: 오차 자체를 보정
- ✓ AdaBoost: 오차 데이터에 집중

05 | Boosting

Gradient boosting

- 핵심 아이디어
 - ✓ loss의 gradient를 따라 모델 업데이트
 - ✓ loss를 가장 빠르게 줄이는 방향으로 학습
 - ✓ $g_i = \frac{\partial \ell}{\partial f(x_i)}$
- pseudo residual
 - ✓ $-g_i$ = loss를 줄이는 방향 (negative gradient)
 - ✓ 새로운 모델은 이 값을 학습
 - ✓ 모델이 “어디를 얼마나 틀렸는지”를 gradient로 표현
- 업데이트 구조
 - ✓ gradient 방향으로 업데이트
- 특징
 - ✓ 다양한 loss 함수 적용 가능
 - ✓ Boosting의 일반화된 형태

05 | Boosting

Gradient tree boosting & XGBoost

- Gradient Tree Boosting
 - ✓ weak learner로 regression tree 사용
 - ✓ gradient(오차 방향)를 학습
 - ✓ 입력 공간을 여러 영역으로 분할
 - ✓ 각 leaf에서 residual 평균으로 예측값 보정
- XGBoost
 - ✓ 미분 가능한 손실함수 기반으로 gradient와 Hessian을 이용해 최적화
 - ✓ $L(f) = \sum_{i=1}^N \ell(y_i, f(x_i)) + \Omega(f)$
 - ✓ $\Omega(f) = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2$
- XGBoost 주요 특징
 - ✓ 정규화: tree 복잡도 제어 $(\gamma, \lambda) \Rightarrow$ 과적합 감소
 - ✓ 2차 미분 (Hessian): gradient + curvature $\Rightarrow$ 정확한 업데이트
 - ✓ gain 기반 split: loss 감소량 기준 최적 분할

06 | Interpreting tree importance

Feature importance

- Feature importance 개념
 - ✓ 모델이 어떤 feature를 얼마나 중요하게 사용하는지 측정
 - ✓ tree 기반 모델에서 split 성능 개선량(gain) 기준으로 정의
- 단일 트리 기준
 - ✓ $R_k(T) = \sum G_j \cdot I(v_j = k)$
 - ✓ feature k를 사용한 모든 split에서의 gain 합
 - ✓ G_j : 해당 노드에서 loss 감소량
- 앙상블 기준
 - ✓ $R_k = \frac{1}{M} \sum_{m=1}^M R_k(T_m)$
 - ✓ 여러 트리 평균으로 안정적인 중요도 계산
- 해석
 - ✓ 값이 클수록 예측에 크게 기여
 - ✓ normalize하여 상대적 중요도 비교
 - ✓ “이 feature 제거 시 성능 저하 큼” 의미

06 | Interpreting tree importance

Partial dependency plot (PDP)

- PDP 목적

- ✓ 특정 feature가 예측에 미치는 영향 분석
- ✓ 다른 feature 영향 제거 (marginalization)

- 정의

- ✓ $f_k(x_k) = \frac{1}{N} \sum f(x_{-k}, x_k)$
- ✓ x_k : 관심 feature
- ✓ x_{-k} : 나머지 feature

- 동작 방식

- ✓ 하나의 feature 값만 변화시킴
- ✓ 나머지 feature는 데이터 평균 효과로 처리

- 해석

- ✓ x축: feature 값
- ✓ y축: 모델 출력 변화

⇒ feature 증가/감소가 예측에 미치는 영향 확인

- 확장 (interaction)

- ✓ $f_{jk}(x_j, x_k)$
- ✓ 두 feature의 상호작용 효과 분석 가능



Q&A

감사합니다.